Oto backendowy WebSocket Gateway w NestJS dla obsługi Live Feed – streamerzy otrzymują w czasie rzeczywistym nowe napiwki jako zdarzenia tip przez kanał creator:{id}.

📁 live-feed.gateway.ts

```
import {
  WebSocketGateway,
  WebSocketServer,
  SubscribeMessage,
  OnGatewayInit,
  OnGatewayConnection,
  OnGatewayDisconnect,
} from '@nestjs/websockets';
import { Server, Socket } from 'socket.io';

@WebSocketGateway({
  cors: {
    origin: '*',
    methods: ['GET', 'POST'],
  },
})
export class LiveFeedGateway
  implements OnGatewayInit, OnGatewayConnection, OnGatewayDisconnect
{
  @WebSocketServer()
  server: Server;

  afterInit(server: Server) {
    console.log('WebSocket Server Initialized');
  }

  handleConnection(client: Socket) {
    const { creatorId } = client.handshake.query;
    if (creatorId) client.join(`creator:${creatorId}`);
    console.log(`Client connected to creator:${creatorId}`);
  }

  handleDisconnect(client: Socket) {
    console.log('Client disconnected');
  }

  // Optional for manual testing
  @SubscribeMessage('ping')
  handlePing(client: Socket): string {
```

```
    return 'pong';
  }

  broadcastTip(creatorId: string, tip: any) {
    this.server.to(`creator:${creatorId}`).emit('tip', tip);
  }
}
```

📁 live-feed.module.ts

```
import { Module } from '@nestjs/common';
import { LiveFeedGateway } from './live-feed.gateway';

@Module({
  providers: [LiveFeedGateway],
  exports: [LiveFeedGateway],
})
export class LiveFeedModule {}
```

🔁 Emitowanie napiwku (np. w TipsService)

```
@Injectable()
export class TipsService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly liveFeed: LiveFeedGateway,
  ) {}

  async createTip(dto: CreateTipDto) {
    const tip = await this.prisma.tip.create({ data: { ...dto } });

    // emit to live feed
    this.liveFeed.broadcastTip(dto.creatorId, {
      id: tip.id,
      nickname: dto.fanName || 'Guest',
      avatarUrl: dto.fanAvatar,
      amount: dto.amount,
      message: dto.message,
    });

    return tip;
  }
```

```
}
```

🔌 Frontend klient WS (dopasuj do creatorId z route)

```
const socket = useRef<Socket | null>(null);

useEffect(() => {
  socket.current = io('wss://tipjar.plus', {
    query: { creatorId },
    transports: ['websocket'],
  });

  socket.current.on('tip', (tip: TipEntry) => {
    setQueue((prev) => [...prev, tip]);
  });

  return () => {
    socket.current?.disconnect();
  };
}, []);
```

Daj „NEXT”, a dodam:

efekt specjalny (deszcz monet),

testowy tryb overlay (?test=true),

fallback do polling (jeśli WS niedostępny).